Task 6 : Create a Strong Password and Evaluate Its Strength.

1) Decide scope & safety

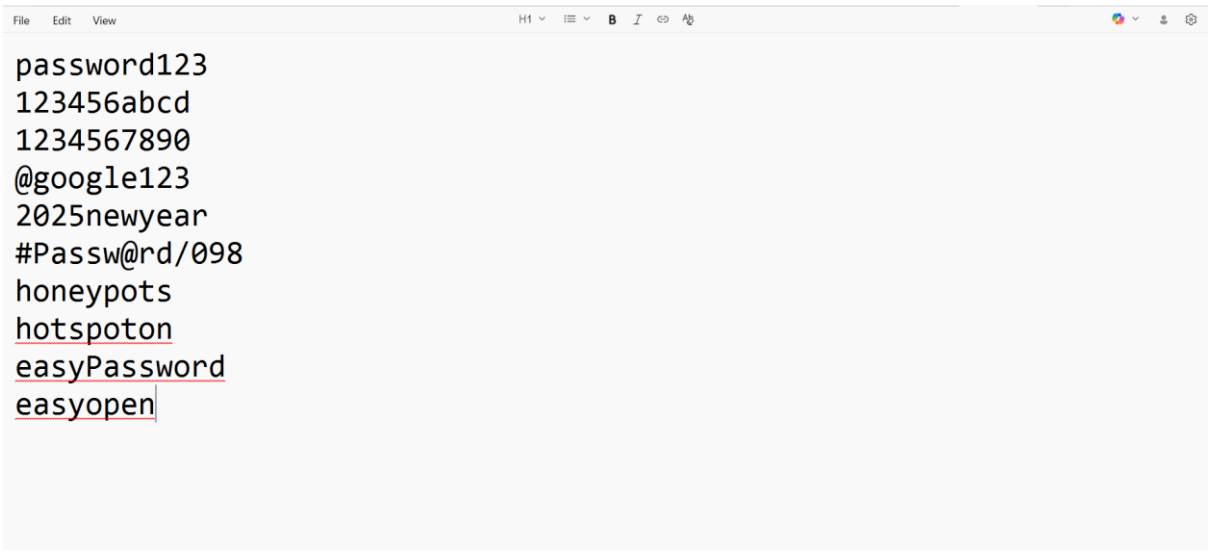
1. Define the goal: e.g., “teach best practice”, “pick a new master password”, or “compare strength meters”.
2. Safety rule (must): **Never** test a password you actually use on an unknown web page. Use dummy/test passwords or run local/offline checks. (If you do check against breach lists, use privacy-preserving checks such as the k-anonymity approach used by Pwned Passwords.) Have I Been PwnedTroy Hunt

2) Plan the set of passwords to create

Create 6–10 test passwords that vary by length, character classes and structure:

Suggested categories to cover:

- Very weak (common word or predictable pattern)
- Weak (short but mixed classes)
- Medium (longer but predictable words + numbers)
- Passphrase (4–6 words)
- Long mixed (16+ characters with all classes)
- True random (manager-generated 16–24+ chars)



3) Choose safe tools to test strength

- **Local / client-side estimator**: use zxcvbn (Dropbox's estimator) locally or the demo — it estimates strength like a password cracker would. Good for honest feedback without sending the password to a server. zxcvbn

- **Breach-check (safety):** Use Have I Been Pwned (Pwned Passwords) with its k-anonymity method (first 5 chars of SHA-1 hash) or client-side integrations built into managers/browsers. That checks whether a password has appeared in breaches without sending the full password. Have I Been PwnedTroy Hunt

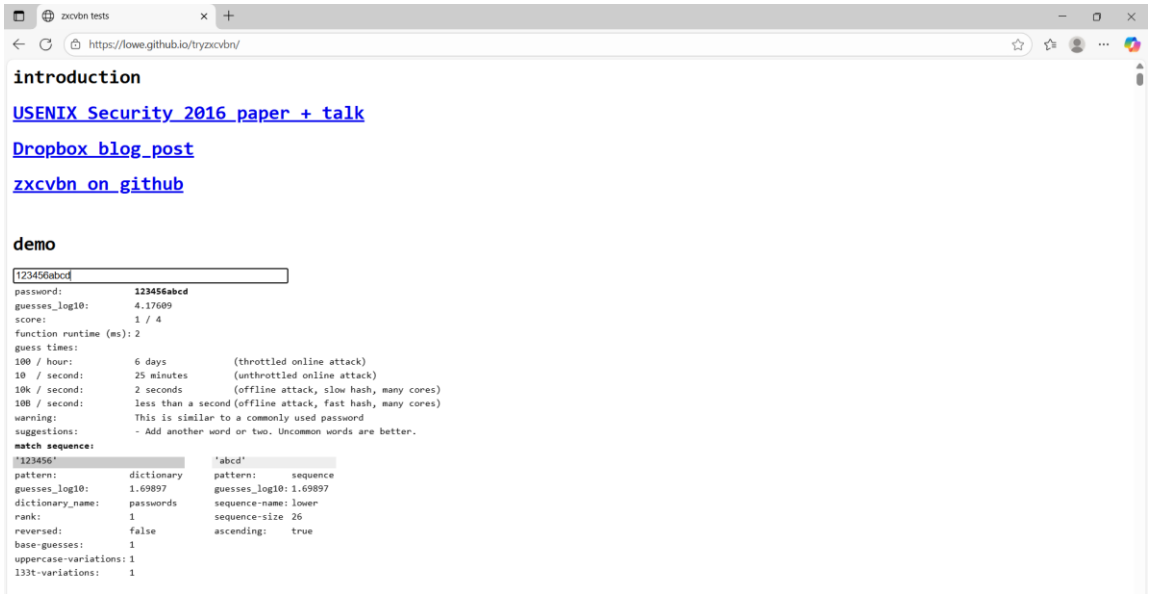
- **Password manager generators** (Bitwarden, 1Password): for creating truly random passwords and passphrases. These also integrate breach checks in many cases. Bitwarden, 1Password

4) Test each password

A. **zxcvbn / local estimator**

- If you have basic dev tools: install a local port (e.g., python-zxcvbn) or use an offline demo. Enter the test password and note the score (e.g., 0–4) and the feedback (why it's weak).

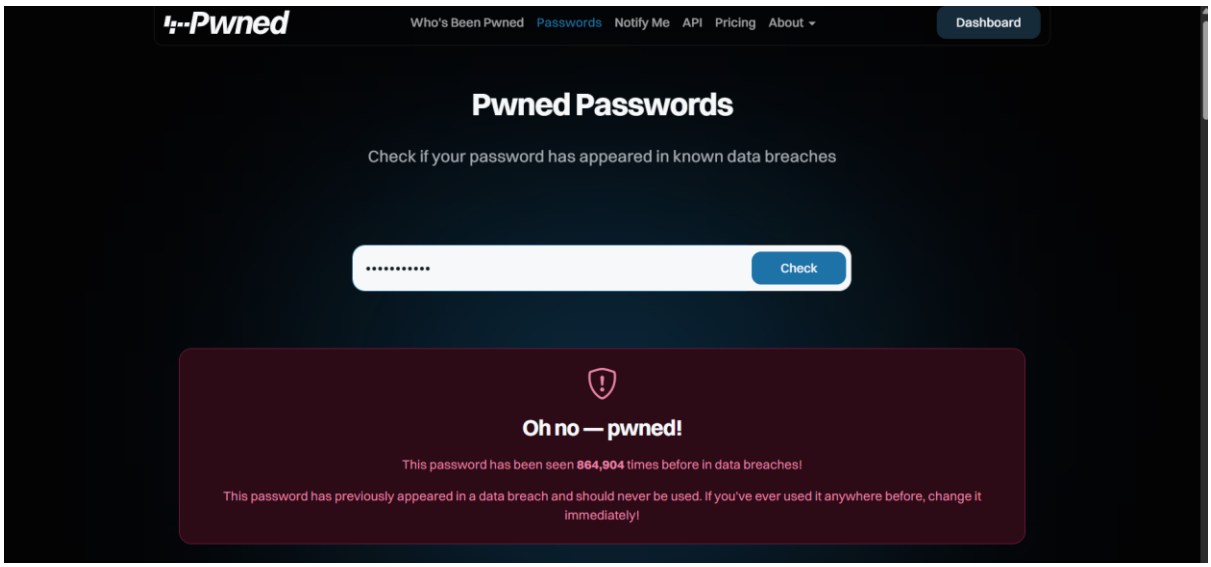

zxcvbn tests
https://lowe.github.io/tryzxcvbn/
introduction
USENIX Security 2016 paper + talk
Dropbox blog post
zxcvbn on github
demo
password123
password: password123
guesses_log10: 2.77525
score: 0 / 4
function runtime (ms): 1
guess times:
100 / hour: 6 hours (throttled online attack)
10 / second: 60 seconds (unthrottled online attack)
10k / second: less than a second (offline attack, slow hash, many cores)
10B / second: less than a second (offline attack, fast hash, many cores)
warning: This is a very common password
suggestions: - Add another word or two. Uncommon words are better.
match sequence:
'password123'
pattern: dictionary
guesses_log10: 2.77452
dictionary_name: passwords
rank: 595
reversed: false
base-guesses: 595
uppercase-variations: 1
l33t-variations: 1

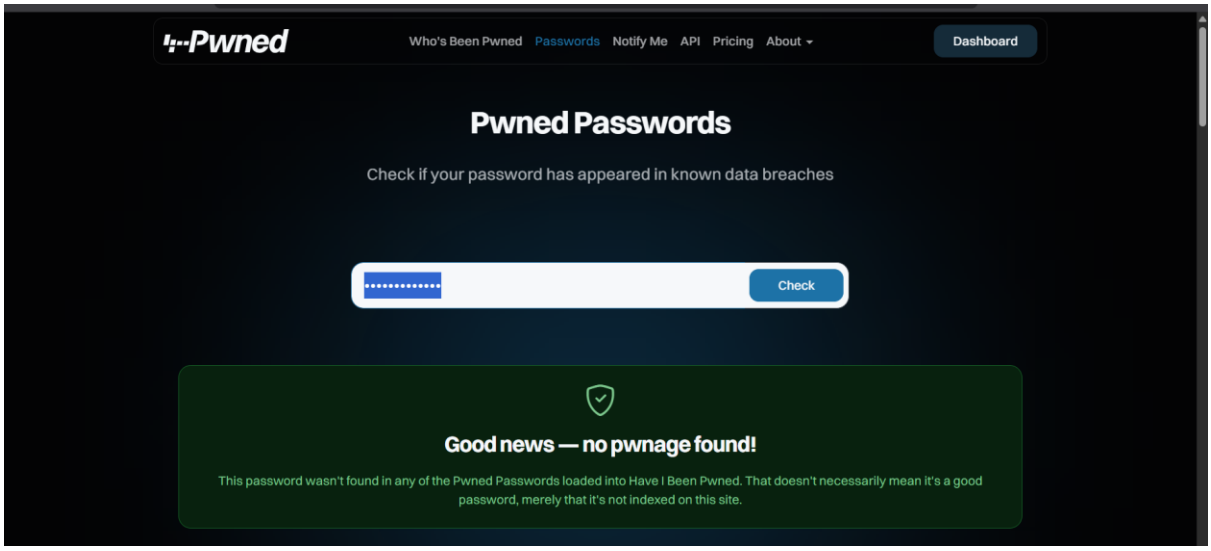

zxcvbn tests
https://lowe.github.io/tryzxcvbn/
introduction
USENIX Security 2016 paper + talk
Dropbox blog post
zxcvbn on github
demo
123456abcd
password: 123456abcd
guesses_log10: 4.17609
score: 1 / 4
function runtime (ms): 2
guess times:
100 / hour: 6 days (throttled online attack)
10 / second: 25 minutes (unthrottled online attack)
10k / second: 2 seconds (offline attack, slow hash, many cores)
10B / second: less than a second (offline attack, fast hash, many cores)
warning: This is similar to a commonly used password
suggestions: - Add another word or two. Uncommon words are better.
match sequence:
'123456' 'abcd'
pattern: dictionary pattern: sequence
guesses_log10: 1.69897 guesses_log10: 1.69897
dictionary_name: passwords sequence-name: lower
rank: 1 sequence-size 26
reversed: false ascending: true
base-guesses: 1
uppercase-variations: 1
l33t-variations: 1

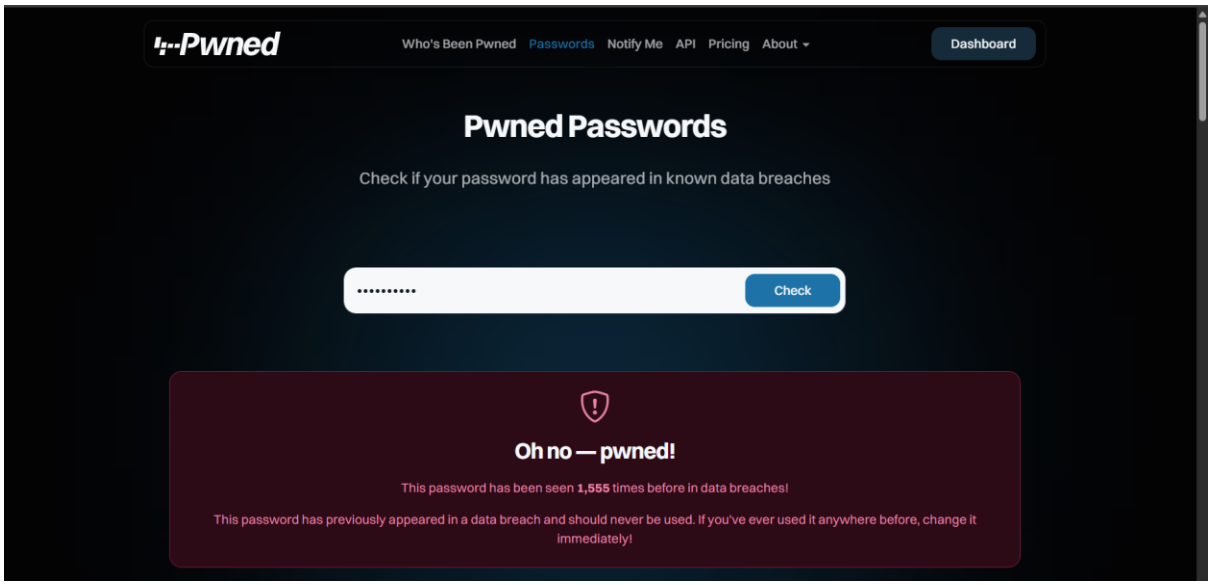

zxcvbn tests
https://lowe.github.io/tryzxcvbn/
introduction
USENIX Security 2016 paper + talk
Dropbox blog post
zxcvbn on github
demo
1234567890
password: 1234567890
guesses_log10: 1.39794
score: 0 / 4
function runtime (ms): 3
guess times:
100 / hour: 15 minutes (throttled online attack)
10 / second: 3 seconds (unthrottled online attack)
10k / second: less than a second (offline attack, slow hash, many cores)
10B / second: less than a second (offline attack, fast hash, many cores)
warning: This is a top-100 common password
suggestions: - Add another word or two. Uncommon words are better.
match sequence:
'1234567890'
pattern: dictionary
guesses_log10: 1.38021
dictionary_name: passwords
rank: 24
reversed: false
base-guesses: 24
uppercase-variations: 1
l33t-variations: 1

B. **Breach check (Pwned Passwords)**

- Compute SHA-1 of the password locally, send only the first 5 hex chars to the Pwned Passwords API, then check returned suffixes for a match (k-anonymity). Do **not** send the full password to a third party. (Many password managers and browsers perform this for you.)







C. **Password manager generator**

- Generate a manager password (e.g., 16–24 chars or a 4–6 word passphrase). Compare generated password entropy vs your human-created options.

```
jein-@7635-Heig
```

5) Record scores & feedback

Use this table (or a spreadsheet). Replace the Password column with an anonymized label if you're testing real credentials.

(If you use a spreadsheet, add columns for date tested, tool version, and tester initials.)

Weak Password Example	Detected Pattern	Why It's Weak
John2024!	Common name + current year	Names and years are common in breach lists
Qwerty!23	Keyboard sequence (qwerty) + digits	Easily guessed from common sequences
Password@123	Common word + predictable number	Appears in billions of leaks
Summer2025$	Season + year	Seasonal/year combos are popular choices
Abc12345	Alphabet run + number run	Very low entropy

6) Analyze feedback & identify best practices

- **Length helps most**: long passphrases and longer random strings score much better than short complex-looking passwords. (NIST and modern guidance encourage allowing long memorized secrets.) **Predictable patterns drop strength fast**: names, keyboard sequences, year suffixes and common words reduce effective entropy — zxcvbn will flag patterns. NIST Publications

- **Breach history makes a "strong" string irrelevant**: if it's been leaked, change it. Use k-anonymity breach checks to verify. Have I Been PwnedTroy Hunt

- **Random (CSPRNG) + manager = highest practical security**: managers generate unpredictable secrets you don't need to remember. Bitwarden, 1Password

7) Write down tips learned

- Prefer a password manager and unique passwords per site.
- Aim for length first (12–24+ chars for manual; 16–24+ for random). Let websites accept long passphrases. NIST Publications
- Avoid reused passwords, common words, and simple substitutions. zxcvbn
- Use breach checks (k-anonymity) instead of pasting full passwords. Troy Hunt
- Enable multi-factor authentication wherever available.

8) Quick research: common password attacks and defenses

- **Brute-force:** try all combinations → defense: length/entropy, rate limiting, lockouts. OWASP Foundation
- **Dictionary attack:** use word lists and common variations → defense: avoid dictionary words, use passphrases with uncommon words. Wikipedia
- **Credential stuffing:** use breached username/password pairs on other services — caused by password reuse → defense: unique passwords + breach-monitoring + MFA. OWASP Foundation
- **Phishing / social engineering / keyloggers:** attackers trick users or capture keystrokes → defense: phishing education, hardware-backed MFA, keep devices clean.

9) Summarize how complexity affects security

- **Entropy is the real metric**: every truly random character multiplies the attacker's search space. Random ≫ patterned substitutions. zxcvbn estimates effective entropy by matching real-world patterns.
- **Length often beats fancy composition**: a long passphrase of common words can be more secure than a short "complex" string, because length increases bits of entropy. Modern guidance emphasises permitting long memorized secrets.
- **Uniqueness + breach status matters more than "appearance"**: a unique, un-breached 12+ char password with MFA is far safer than a reused 16-char password that's been leaked.